ГАЙД ДЛЯ НАЧИНАЮЩИХ · GOLANG

Трёхслойная архитектура в Go

handler → service → repository + models

Что делает каждый слой, за что он отвечает и за что — нет.
Простым языком, с аналогиями, полным примером кода и тестом.

internal/handler/

internal/service/

internal/repository/

internal/models/

Примеры написаны на стандартной библиотеке Go 1.22+ (net/http, database/sql)

Содержание

1. Зачем вообще нужна архитектура . . . 3
2. Аналогия: ресторан . . . 4
3. Общая картина . . . 5
4. Структура проекта . . . 6
5. Слой Models — данные . . . 8
6. Слой Repository — хранилище . . . 10
7. Слой Service — бизнес-логика . . . 13
8. Слой Handler — общение с миром . . . 16
9. Собираем всё в main.go . . . 19
10. Путь одного запроса . . . 20
11. Интерфейсы и тестирование . . . 22
12. Как ошибки проходят через слои . . . 24
13. Частые ошибки новичков . . . 25
14. Шпаргалка: что куда положить . . . 26
15. Задания для практики . . . 27

Главная идея в одном абзаце

Если запомнить из всего гайда только одну мысль, пусть будет эта: **каждый слой отвечает на свой собственный вопрос и не лезет в чужие дела.**

Handler — *«Как общаться с внешним миром?»*
Service — *«Что и по каким правилам должно произойти?»*
Repository — *«Где и как хранятся данные?»*
Models — *«Как выглядят данные, о которых мы говорим?»*

1. Зачем вообще нужна архитектура

Когда пишешь первый веб-сервер на Go, проще всего засунуть всё в одну функцию: прочитать JSON, проверить данные, сходить в базу, отправить ответ. Для учебной задачи на 30 строк это работает. Давай посмотрим, как это выглядит:

спагетти-код: main.go

```go
// Так делать НЕ надо: всё в одной куче
func createUser(w http.ResponseWriter, r *http.Request) {
    var req struct {
        Name  string `json:"name"`
        Email string `json:"email"`
    }
    json.NewDecoder(r.Body).Decode(&req)

    if req.Name == "" {
        http.Error(w, "name is required", 400)
        return
    }

    var exists bool
    db.QueryRow("SELECT EXISTS(SELECT 1 FROM users WHERE email=$1)",
        req.Email).Scan(&exists)
    if exists {
        http.Error(w, "email taken", 409)
        return
    }

    var id int64
    db.QueryRow("INSERT INTO users (name, email) VALUES ($1, $2) RETURNING id",
        req.Name, req.Email).Scan(&id)

    json.NewEncoder(w).Encode(map[string]any{"id": id})
}
```

Код рабочий. Но как только проект подрастёт, начнутся проблемы:

- **Нельзя протестировать логику без базы данных.** Хочешь проверить правило «email должен быть уникальным» — поднимай PostgreSQL.
- **Нельзя переиспользовать.** Завтра попросят регистрацию через Telegram-бота или CLI — придётся копировать половину кода, потому что логика склеена с HTTP.
- **Сложно менять.** Решили перейти с PostgreSQL на MongoDB — переписываешь все хендлеры, хотя бизнес-правила не изменились.
- **Сложно читать.** В одной функции перемешаны HTTP, правила и SQL. В проекте из 50 таких функций новичок в команде просто утонет.
- **Глобальная переменная** `db` — её не подменить в тестах, и непонятно, кто и где ею пользуется.

Решение — **разделить код по зонам ответственности**. Это и есть слоистая архитектура. В Go-сообществе самый популярный и понятный её вариант — **handler → service → repository** плюс **models** как общий «словарь» данных. Её часто называют трёхслойной: три слоя «работают», а models — это общие структуры, которыми они обмениваются.

Зачем это лично тебе

Архитектура — это не про «красиво» и не про «как в больших компаниях». Это про то, чтобы **изменение в одном месте не ломало всё остальное**. Поменял базу — трогаешь только repository. Поменял правило регистрации — только service. Поменял формат API — только handler.

2. Аналогия: ресторан

Самый простой способ понять слои — представить ресторан. Там тоже у каждого своя работа, и никто не делает чужую.

Слой	Кто в ресторане	Что делает	Чего НЕ делает
Handler	Официант	Принимает заказ у гостя, записывает его понятным для кухни языком, приносит готовое блюдо или говорит «извините, этого нет».	Не готовит еду. Не ходит на склад. Не решает, как жарить стейк.
Service	Шеф-повар	Знает рецепты и правила: «стейк medium — 4 минуты», «без лука нельзя, это часть рецепта». Решает, можно ли выполнить заказ.	Не выходит в зал к гостям. Не знает, в каком холодильнике лежит мясо и у какого поставщика его купили.
Repository	Кладовщик	Хранит продукты, выдаёт по запросу, принимает поставки. Знает, на какой полке что лежит.	Не знает рецептов. Не решает, что готовить. Просто «дай» и «положи».
Models	Продукты и блюда	То, что передаётся между всеми: «стейк», «заказ №12», «картошка». Все понимают, о чём речь.	Сами ничего не делают — это просто «вещи», о которых все говорят.

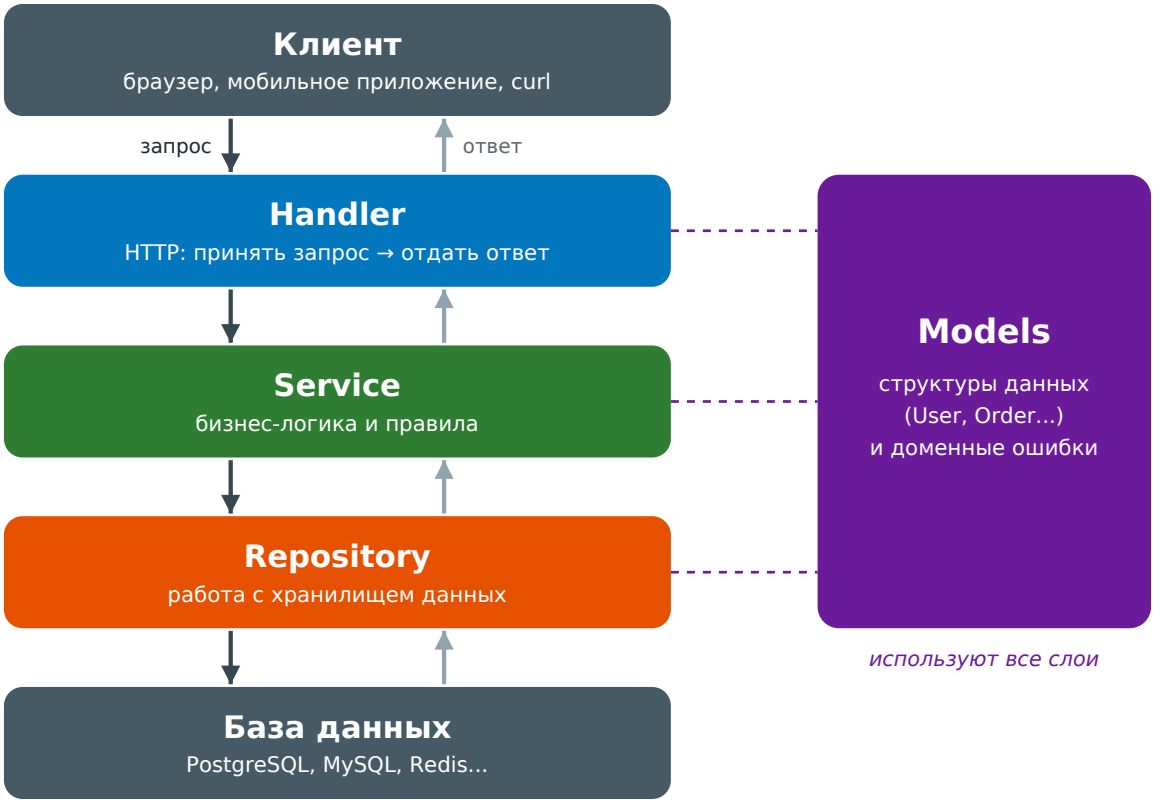
Аналогия

Если кладовщик заболеет и его заменят другим (или склад переедет в другое здание) — повару всё равно: он просто говорит «дай 2 кг картошки». Ровно так же сервису всё равно, PostgreSQL у тебя или MongoDB: он просит репозиторий `GetByID(42)` и получает пользователя.

А если вместо официанта заказ примет курьер-доставщик (Telegram-бот вместо HTTP), повар будет готовить по тем же рецептам. Это и есть переиспользование сервиса.

3. Общая картина

Вот как запрос проходит через слои. Запрос идёт сверху вниз, ответ поднимается снизу вверх. Models — сбоку, потому что ими пользуются все.



Правило направления зависимостей

Это самое важное правило всей архитектуры. **Слой может знать только о слоях ниже себя и никогда — о слоях выше.**

- **Handler** знает о service (вызывает его) и о models.
- **Service** знает о repository (через интерфейс) и о models. Про HTTP **не знает ничего**.
- **Repository** знает о базе данных и о models. Про service и handler **не знает ничего**.
- **Models** не знает **ни о ком**. Это самый «чистый» пакет, он ничего не импортирует из проекта.

Запомни

Если в пакете service появился импорт **net/http**, а в repository — импорт пакета **service**, значит что-то пошло не так. Проверяй импорты — это самый быстрый способ заметить нарушение архитектуры.

Бонус: Go вообще запрещает циклические импорты. Если handler импортирует service, а service импортирует handler — программа просто не скомпилируется.

Слой	Отвечает на вопрос	Знает о	Не знает о
Handler	Как общаться с внешним миром?	HTTP, JSON, статус-коды, service, models	SQL, базе данных, бизнес-правилах
Service	Что должно произойти и по каким правилам?	бизнес-правилах, интерфейсе репозитория, models	HTTP, JSON, SQL, конкретной БД
Repository	Где и как хранятся данные?	SQL, драйвере БД, models	HTTP, бизнес-правилах
Models	Как выглядят данные?	только о себе	обо всём остальном

4. Структура проекта

Весь гайд построен на одном сквозном примере: **небольшой API для пользователей**. Он умеет две вещи: зарегистрировать пользователя (`POST /users`) и получить пользователя по id (`GET /users/{id}`). Этого достаточно, чтобы увидеть работу каждого слоя. Вот как выглядят папки:

дерево проекта

```
myapp/
├── cmd/
│   └── app/
│       └── main.go            ← точка входа: собираем все слои вместе
├── internal/
│   ├── models/
│   │   ├── user.go            ← структура User
│   │   └── errors.go          ← доменные ошибки (ErrUserNotFound и т.д.)
│   ├── repository/
│   │   └── user_postgres.go   ← SQL-запросы к таблице users
│   ├── service/
│   │   ├── user.go            ← бизнес-логика пользователей
│   │   └── user_test.go       ← тесты сервиса (без базы!)
│   └── handler/
│       ├── user.go            ← HTTP-обработчики /users
│       └── response.go        ← хелперы для JSON-ответов
├── migrations/
│   └── 001_create_users.sql   ← создание таблицы
├── go.mod
└── go.sum
```

Что значат эти папки

- `cmd/app/` — здесь лежит `main.go`. Если у проекта будет несколько программ (например, сервер и утилита миграций), у каждой своя папка в `cmd/`.
- `internal/` — особая папка в Go. Код внутри неё **нельзя импортировать из других модулей** — это проверяет сам компилятор. Удобно: ты точно знаешь, что это «внутренности» твоего приложения.
- `migrations/` — SQL-файлы, которые создают и меняют таблицы в базе.

Таблица, с которой будем работать:

migrations/001_create_users.sql

```sql
CREATE TABLE users (
    id         BIGSERIAL PRIMARY KEY,
```

migrations/001_create_users.sql

```
    name       TEXT        NOT NULL,
    email      TEXT        NOT NULL UNIQUE,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Для любопытных

Это не единственный «правильный» вариант раскладки. Иногда делят не по слоям, а по фичам: `internal/user/handler.go`, `internal/user/service.go`, `internal/order/...`. Идея слоёв та же самая, просто папки организованы иначе. Для старта раскладка по слоям — самая наглядная.

5. Слой Models — данные

СЛОЙ 1

Models

«Как выглядят данные, о которых мы говорим?»

Начнём с самого простого слоя, потому что от него зависят все остальные. **Models — это просто структуры Go, которые описывают сущности твоего приложения**: пользователь, заказ, товар, комментарий. Это общий «словарь» проекта: когда handler, service и repository говорят о пользователе, они все имеют в виду одну и ту же структуру `models.User`.

internal/models/user.go

```go
package models

import "time"

// User — пользователь нашего приложения.
// Это просто описание данных, никакой логики работы с БД или HTTP.
type User struct {
    ID        int64     `json:"id"`
    Name      string    `json:"name"`
    Email     string    `json:"email"`
    CreatedAt time.Time `json:"created_at"`
}
```

Теги вида `` `json:"id"` `` подсказывают пакету encoding/json, как называть поля в JSON. Без них поле `CreatedAt` в ответе так и называлось бы `CreatedAt`, а мы хотим `created_at`.

Доменные ошибки

Второе, что удобно хранить в models, — **ошибки предметной области**. «Пользователь не найден», «email уже занят» — это не ошибки PostgreSQL и не ошибки HTTP, это понятия нашего приложения. Раз models не зависит ни от кого, а от models зависят все, то эти ошибки доступны любому слою без циклических импортов.

internal/models/errors.go

```go
package models

import "errors"

// Ошибки, которые понятны всему приложению.
// Repository их возвращает, service их создаёт,
// handler превращает их в HTTP-статусы.
var (
    ErrUserNotFound = errors.New("user not found")
    ErrEmailTaken   = errors.New("email already taken")
    ErrInvalidInput = errors.New("invalid input")
)
```

✓ Зона ответственности (делает)	✗ Не его дело (не делает)
✓ Описывать сущности: структуры с полями.	✗ Импортировать другие пакеты проекта (handler, service, repository).
✓ Хранить доменные ошибки (ErrUserNotFound и т.п.).	✗ Ходить в базу данных.
✓ Иметь простые методы, которые зависят только от полей самой структуры: `func (u User) IsAdult() bool`.	✗ Знать про HTTP, запросы и ответы.
✓ Константы предметной области: статусы заказа, роли пользователей.	✗ Содержать сложную логику, которой нужны внешние данные — это работа service.

Совет

Про json-теги в моделях. В учебных и небольших проектах держать `json`-теги прямо в моделях — нормально. В больших проектах часто делают отдельные структуры для API (их называют **DTO** — Data Transfer Object) в слое handler. Тогда, например, добавив в User поле PasswordHash, ты точно не отправишь его клиенту случайно. Мы в примере используем DTO для входящего запроса, а для ответа — саму модель, чтобы не усложнять.

Про название пакета

В Go-сообществе пакет с именем `models` иногда критикуют: по правилам хорошего тона имя пакета должно говорить, *что* в нём лежит. Поэтому ты можешь встретить названия `domain`, `entity` или `user`. Суть от этого не меняется — это тот же самый слой.

6. Слой Repository — хранилище

СЛОЙ 2

Repository

«Где и как хранятся данные?»

Repository — единственный слой, который знает, где лежат данные и как с ними работать. Только здесь пишется SQL, только здесь знают про таблицы, колонки, драйвер базы. Для всего остального приложения репозиторий — «чёрный ящик» с понятными кнопками: «сохрани пользователя», «найди пользователя по id», «найди по email».

Репозиторий берёт строки из базы и превращает их в модели (и обратно). Наружу он отдаёт `models.User`, а не строки таблицы и не `sql.Rows`.

internal/repository/user_postgres.go

```go
package repository

import (
    "context"
    "database/sql"
    "errors"
    "fmt"

    "myapp/internal/models"
)

// UserRepository работает с таблицей users в PostgreSQL.
type UserRepository struct {
    db *sql.DB
}

// NewUserRepository — конструктор. Подключение к БД приходит снаружи.
func NewUserRepository(db *sql.DB) *UserRepository {
    return &UserRepository{db: db}
}

// Create сохраняет пользователя. ID и CreatedAt генерирует база,
// поэтому мы получаем их через RETURNING и записываем в ту же структуру.
func (r *UserRepository) Create(ctx context.Context, u *models.User) error {
    query := `INSERT INTO users (name, email)
              VALUES ($1, $2)
              RETURNING id, created_at`

    err := r.db.QueryRowContext(ctx, query, u.Name, u.Email).
        Scan(&u.ID, &u.CreatedAt)
    if err != nil {
        return fmt.Errorf("insert user: %w", err)
    }
    return nil
}

// GetByID ищет пользователя по id.
func (r *UserRepository) GetByID(ctx context.Context, id int64) (*models.User, error) {
    query := `SELECT id, name, email, created_at FROM users WHERE id = $1`

    var u models.User
    err := r.db.QueryRowContext(ctx, query, id).
        Scan(&u.ID, &u.Name, &u.Email, &u.CreatedAt)
    if errors.Is(err, sql.ErrNoRows) {
        // Переводим "ошибку базы" в "ошибку нашего приложения"
```

internal/repository/user_postgres.go

```
        return nil, models.ErrUserNotFound
    }
    if err != nil {
        return nil, fmt.Errorf("select user by id: %w", err)
    }
    return &u, nil
}

// GetByEmail ищет пользователя по email.
func (r *UserRepository) GetByEmail(
    ctx context.Context, email string,
) (*models.User, error) {
    query := `SELECT id, name, email, created_at FROM users WHERE email = $1`

    var u models.User
    err := r.db.QueryRowContext(ctx, query, email).
        Scan(&u.ID, &u.Name, &u.Email, &u.CreatedAt)
    if errors.Is(err, sql.ErrNoRows) {
        return nil, models.ErrUserNotFound
    }
    if err != nil {
        return nil, fmt.Errorf("select user by email: %w", err)
    }
    return &u, nil
}
```

Разбираем по шагам

- **Структура с полем `db`.** Подключение к базе не глобальная переменная, а поле структуры. Его передают через конструктор — так понятно, откуда оно взялось, и его легко подменить.
- **Плейсхолдеры `$1`, `$2`.** Значения передаются отдельно от текста запроса. Никогда не собирай SQL через `fmt.Sprintf` или сложение строк — это прямой путь к SQL-инъекциям.
- **Перевод ошибок.** `sql.ErrNoRows` — это «язык» пакета database/sql. Наверх мы отдаём `models.ErrUserNotFound` — «язык» нашего приложения. Благодаря этому сервис и handler вообще не знают, что внутри SQL.
- **Оборачивание ошибок через `%w`.** Неожиданные ошибки (упала сеть, сломался запрос) мы оборачиваем с пояснением, где они случились. Так в логах будет видно `select user by id: connection refused`, а не просто `connection refused`.
- `*models.User` **в Create.** Мы передаём указатель, чтобы репозиторий мог дописать в ту же структуру ID и дату, которые сгенерировала база.

Что за ctx context.Context?

`context.Context` — это «паспорт запроса». Он несёт с собой сигнал отмены и дедлайн. Если клиент закрыл вкладку браузера или прошёл таймаут, контекст отменяется, и запрос к базе тоже прерывается, а не висит впустую.

Правило простое: `ctx` — **всегда первый аргумент** функции, и его передают по цепочке handler → service → repository. В handler его берут из запроса: `r.Context()`. Сам контекст в структуры не сохраняют.

✓ Зона ответственности (делает)	✗ Не его дело (не делает)
✓ Писать SQL-запросы (или запросы к Mongo, Redis, файлам, внешнему API-хранилищу).	✗ Решать бизнес-вопросы: «можно ли регистрировать?», «хватает ли денег?».
✓ Превращать строки БД в модели и обратно.	✗ Проверять пользовательский ввод (длина имени, формат email).
✓ Переводить ошибки хранилища в доменные: `sql.ErrNoRows` → `ErrUserNotFound`.	✗ Знать про HTTP, JSON-запросы, статус-коды.
✓ Транзакции внутри одной операции хранилища.	✗ Вызывать сервисы или другие слои выше себя.

Аналогия

Репозиторий — это кладовщик. Ему говорят «дай пользователя №42» — он идёт на нужную полку и приносит. Если пользователя нет — говорит «нет такого». Но кладовщик не решает, *стоит ли* выдавать этого пользователя и *что* с ним потом делать.

7. Слой Service — бизнес-логика

слой 3

Service

«Что должно произойти и по каким правилам?»

Service — сердце приложения. Здесь живут бизнес-правила: всё то, что заказчик или менеджер объяснил бы тебе обычными словами, без слов «HTTP» и «SQL»:

- «Имя пользователя не может быть пустым».
- «Два пользователя не могут иметь один и тот же email».
- «Email не зависит от регистра: Anya@Mail.ru и anya@mail.ru — один и тот же адрес».
- «Нельзя заказать больше 10 товаров», «после регистрации отправить письмо» и т.д.

Обрати внимание: ни одно из этих правил не говорит, *как* пришёл запрос и *где* хранятся данные. Именно поэтому сервис ничего не знает ни про HTTP, ни про SQL.

internal/service/user.go

```go
package service

import (
    "context"
    "errors"
    "fmt"
    "strings"

    "myapp/internal/models"
)

// UserRepository — то, что сервису НУЖНО от хранилища.
// Интерфейс объявлен здесь, в слое, который им пользуется.
type UserRepository interface {
    Create(ctx context.Context, u *models.User) error
    GetByID(ctx context.Context, id int64) (*models.User, error)
    GetByEmail(ctx context.Context, email string) (*models.User, error)
}

type UserService struct {
    repo UserRepository
}

func NewUserService(repo UserRepository) *UserService {
    return &UserService{repo: repo}
}

// Register регистрирует нового пользователя по бизнес-правилам.
func (s *UserService) Register(
    ctx context.Context, name, email string,
) (*models.User, error) {
    // 1. Нормализация: убираем пробелы, email приводим к нижнему регистру
    name = strings.TrimSpace(name)
    email = strings.ToLower(strings.TrimSpace(email))

    // 2. Бизнес-валидация
    if name == "" {
        return nil, fmt.Errorf("%w: name is required", models.ErrInvalidInput)
    }
    if !strings.Contains(email, "@") {
        return nil, fmt.Errorf("%w: invalid email", models.ErrInvalidInput)
    }
```

internal/service/user.go

```
    // 3. Правило: email должен быть уникальным
    existing, err := s.repo.GetByEmail(ctx, email)
    if err != nil && !errors.Is(err, models.ErrUserNotFound) {
        return nil, fmt.Errorf("check email: %w", err)
    }
    if existing != nil {
        return nil, models.ErrEmailTaken
    }

    // 4. Сохраняем
    user := &models.User{Name: name, Email: email}
    if err := s.repo.Create(ctx, user); err != nil {
        return nil, fmt.Errorf("create user: %w", err)
    }
    return user, nil
}

// GetUser возвращает пользователя по id.
func (s *UserService) GetUser(ctx context.Context, id int64) (*models.User, error) {
    if id <= 0 {
        return nil, fmt.Errorf("%w: id must be positive", models.ErrInvalidInput)
    }
    return s.repo.GetByID(ctx, id)
}
```

Самое интересное: интерфейс UserRepository

Заметь: сервис не импортирует пакет `repository`! Вместо этого он сам объявляет интерфейс `UserRepository` — список того, что ему нужно. Сервис как будто говорит: *«Мне нужен кто-то, кто умеет Create, GetByID и GetByEmail. Кто именно — мне всё равно»*.

В Go интерфейсы реализуются **неявно**: не нужно писать «implements». Структура `repository.UserRepository` из прошлого раздела имеет ровно эти три метода с такими же сигнатурами, значит, она автоматически подходит под интерфейс. Что это даёт:

- В тестах можно подсунуть простую «фейковую» реализацию на map, без базы данных (покажу в разделе 11).
- Можно написать реализацию для MongoDB или in-memory — сервис не изменится ни на строчку.
- Сервис зависит от **поведения** («умеет найти пользователя»), а не от **конкретной технологии** («PostgreSQL»).

Запомни

Это знаменитое правило Go: **«Принимай интерфейсы, возвращай структуры»** (accept interfaces, return structs). Конструктор NewUserService принимает интерфейс, а возвращает конкретный *UserService.

И второе правило: **интерфейс объявляют там, где его используют**, а не там, где его реализуют. Поэтому интерфейс репозитория лежит в пакете service.

✓ Зона ответственности (делает)	✗ Не его дело (не делает)
✓ Бизнес-правила и бизнес-валидация.	✗ Знать про `http.Request`, `http.ResponseWriter`, статус-коды.
✓ Нормализация данных (trim, lower-case).	✗ Читать или писать JSON.
✓ Координация: вызвать несколько репозиториев, решить порядок действий.	✗ Писать SQL и знать, какая база используется.
✓ Хэширование паролей, расчёт цены, проверка прав доступа.	✗ Импортировать пакет `handler` или конкретный пакет `repository`.
✓ Вызов внешних сервисов (почта, оплата) — тоже через интерфейсы.	✗ Решать, что ответить клиенту: «404» или «409» — это решает handler.
✓ Возврат доменных ошибок из models.	

Простой тест «правильный ли у меня сервис»

Представь, что завтра твоё приложение будет работать не через HTTP, а как Telegram-бот или консольная утилита. **Если сервис можно использовать без единого изменения — ты всё сделал правильно.** Если придётся его переписывать (потому что он возвращает статус-коды или читает JSON), значит в нём есть чужой код.

Тонкий момент (на будущее)

Проверка «email не занят» в сервисе не спасёт, если два запроса придут одновременно: оба проверят, оба увидят «свободно», оба попробуют вставить. Поэтому в таблице стоит `UNIQUE` на email — база выступает последней линией защиты. Продвинутый вариант: в репозитории распознать ошибку нарушения уникальности и тоже превратить её в `models.ErrEmailTaken`. Для начала достаточно просто знать об этом.

8. Слой Handler — общение с миром

СЛОЙ 4

Handler

«Как общаться с внешним миром?»

Handler — это «пограничник» между внешним миром и твоим приложением. Снаружи он говорит на языке HTTP (URL, JSON, заголовки, статус-коды), а внутрь передаёт обычные Go-значения: строки, числа, структуры. У хорошего хендлера всегда одна и та же работа из четырёх шагов:

1. **Прочитать запрос:** JSON из тела, параметры из пути и query-строки, заголовки.
2. **Проверить формат:** JSON вообще валидный? id — это число?
3. **Вызвать сервис** с уже обычными Go-значениями.
4. **Превратить результат в HTTP-ответ:** успех → статус + JSON, ошибка → подходящий статус + понятное сообщение.

Сначала маленькие хелперы, чтобы не повторять один и тот же код:

internal/handler/response.go

```go
package handler

import (
    "encoding/json"
    "log"
    "net/http"
)

// writeJSON отправляет ответ в формате JSON с нужным статусом.
func writeJSON(w http.ResponseWriter, status int, data any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)
    if err := json.NewEncoder(w).Encode(data); err != nil {
        log.Printf("write response: %v", err)
    }
}

// writeError отправляет ошибку в едином формате: {"error": "..."}
func writeError(w http.ResponseWriter, status int, msg string) {
    writeJSON(w, status, map[string]string{"error": msg})
}
```

А теперь сам хендлер пользователей:

internal/handler/user.go

```go
package handler

import (
    "context"
    "encoding/json"
    "errors"
    "log"
    "net/http"
    "strconv"

    "myapp/internal/models"
)
```

internal/handler/user.go

```go
// UserService — то, что хендлеру нужно от сервиса.
type UserService interface {
    Register(ctx context.Context, name, email string) (*models.User, error)
    GetUser(ctx context.Context, id int64) (*models.User, error)
}

type UserHandler struct {
    service UserService
}

func NewUserHandler(s UserService) *UserHandler {
    return &UserHandler{service: s}
}

// RegisterRoutes подключает маршруты к роутеру (синтаксис Go 1.22+).
func (h *UserHandler) RegisterRoutes(mux *http.ServeMux) {
    mux.HandleFunc("POST /users", h.CreateUser)
    mux.HandleFunc("GET /users/{id}", h.GetUser)
}

// createUserRequest — DTO: как выглядит тело запроса на создание.
type createUserRequest struct {
    Name  string `json:"name"`
    Email string `json:"email"`
}

func (h *UserHandler) CreateUser(w http.ResponseWriter, r *http.Request) {
    // 1. Читаем и проверяем формат
    var req createUserRequest
    if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
        writeError(w, http.StatusBadRequest, "invalid JSON")
        return
    }

    // 2. Вызываем сервис
    user, err := h.service.Register(r.Context(), req.Name, req.Email)
    if err != nil {
        handleError(w, err)
        return
    }

    // 3. Отвечаем
    writeJSON(w, http.StatusCreated, user)
}

func (h *UserHandler) GetUser(w http.ResponseWriter, r *http.Request) {
    id, err := strconv.ParseInt(r.PathValue("id"), 10, 64)
    if err != nil {
        writeError(w, http.StatusBadRequest, "id must be a number")
        return
    }

    user, err := h.service.GetUser(r.Context(), id)
    if err != nil {
        handleError(w, err)
        return
    }
    writeJSON(w, http.StatusOK, user)
}

// handleError — единственное место, где доменная ошибка становится HTTP-статусом.
func handleError(w http.ResponseWriter, err error) {
    switch {
    case errors.Is(err, models.ErrInvalidInput):
        writeError(w, http.StatusBadRequest, err.Error())
    case errors.Is(err, models.ErrUserNotFound):
```

internal/handler/user.go

```
        writeError(w, http.StatusNotFound, "user not found")
    case errors.Is(err, models.ErrEmailTaken):
        writeError(w, http.StatusConflict, "email already taken")
    default:
        // Подробности — в лог для нас, клиенту — общее сообщение
        log.Printf("internal error: %v", err)
        writeError(w, http.StatusInternalServerError, "internal server error")
    }
}
```

Валидация: где проверять что?

Новички часто путаются: «проверку данных делать в handler или в service?». Ответ — **в обоих, но разное**. Handler проверяет **формат** («это вообще можно прочитать?»), service проверяет **смысл** («это допустимо по правилам?»).

Ситуация	Кто проверяет	Почему
Тело запроса — не JSON: `{name: Аня`	**Handler**	Это проблема формата HTTP-запроса. Сервис даже не узнает, что что-то было.
`/users/abc` **— id не число**	**Handler**	Строку из URL нужно превратить в `int64` — это работа «переводчика».
`/users/-5` **— id отрицательный**	**Service**	Формально это число, но по правилам приложения id бывает только положительным.
Пустое имя	**Service**	«Имя обязательно» — бизнес-правило. Оно должно работать и для бота, и для CLI.
Email уже занят	**Service**	Чистое бизнес-правило, и для проверки нужны данные из хранилища.

✓ Зона ответственности (делает)	✗ Не его дело (не делает)
✓ Разбирать запрос: JSON, path-параметры, query, заголовки.	✗ Писать SQL и обращаться к базе напрямую.
✓ Проверять формат входных данных.	✗ Содержать бизнес-правила («email уникален», «возраст от 18»).
✓ Передавать `r.Context()` в сервис.	✗ Вызывать репозиторий в обход сервиса.
✓ Выбирать HTTP-статус по типу ошибки.	✗ Отдавать клиенту текст внутренних ошибок (`pq: relation users...`) — это утечка деталей устройства системы.
✓ Формировать JSON-ответ.	
✓ Логировать внутренние ошибки, не показывая их клиенту.	

Про роутинг

Начиная с Go 1.22 стандартный роутер `http.ServeMux` понимает методы и параметры прямо в шаблоне: `"GET /users/{id}"`. Значение параметра достаётся через `r.PathValue("id")`. Раньше для этого почти всегда брали сторонние роутеры (chi, gorilla/mux) — сейчас для учёбы хватает стандартной библиотеки.

9. Собираем всё в main.go

У нас есть четыре отдельных пакета, которые ничего не знают друг о друге (кроме models). Кто их соединит? `main.go`. Это единственное место, которое знает обо всех слоях сразу, и его задача — **создать все объекты и передать их друг другу**. Собираем снизу вверх:



каждый следующий получает предыдущий в конструктор New...()

cmd/app/main.go

```go
package main

import (
    "database/sql"
    "log"
    "net/http"

    _ "github.com/jackc/pgx/v5/stdlib" // драйвер PostgreSQL для database/sql

    "myapp/internal/handler"
    "myapp/internal/repository"
    "myapp/internal/service"
)

func main() {
    // 1. Подключаемся к базе
    dsn := "postgres://user:password@localhost:5432/myapp?sslmode=disable"
    db, err := sql.Open("pgx", dsn)
    if err != nil {
        log.Fatalf("open db: %v", err)
    }
    defer db.Close()

    if err := db.Ping(); err != nil {
        log.Fatalf("ping db: %v", err)
    }

    // 2. Собираем слои снизу вверх
    userRepo := repository.NewUserRepository(db)
    userService := service.NewUserService(userRepo)
    userHandler := handler.NewUserHandler(userService)

    // 3. Регистрируем маршруты
    mux := http.NewServeMux()
    userHandler.RegisterRoutes(mux)

    // 4. Запускаем сервер
    log.Println("server started on :8080")
    if err := http.ListenAndServe(":8080", mux); err != nil {
        log.Fatal(err)
    }
}
```

Посмотри на строчку `service.NewUserService(userRepo)`. Конструктор ждёт интерфейс `service.UserRepository`, а мы передаём структуру `*repository.UserRepository`. Компилятор проверяет: есть ли у неё все нужные

методы? Есть — значит, всё в порядке. Ровно то же происходит с сервисом и хендлером.

Запомни

То, что мы сделали, называется **Dependency Injection** (внедрение зависимостей). Звучит страшно, но смысл простой: **объект не создаёт свои зависимости сам, а получает их снаружи через конструктор.** Сервис не делает `sql.Open` внутри себя — ему просто дают готовый репозиторий.

В Go это делают руками, как выше, без фреймворков и магии. Для большинства проектов этого более чем достаточно.

Совет

В реальном проекте строку подключения к базе не пишут прямо в коде, а берут из переменной окружения: `os.Getenv("DATABASE_URL")`. Пароли в коде — плохая идея: код попадает в git, а вместе с ним и пароль.

10. Путь одного запроса

Соберём всё вместе и проследим, что происходит, когда клиент отправляет такой запрос:

запрос

```
POST /users
Content-Type: application/json

{"name": "  Аня ", "email": "Anya@Mail.ru"}
```

#	Слой	Что происходит
1	**Handler**	Роутер видит `POST /users` и вызывает `CreateUser`. Хендлер читает JSON в `createUserRequest`. JSON валидный — идём дальше.
2	**Handler**	Вызывает `service.Register(ctx, " Аня ", "Anya@Mail.ru")`. Дальше про HTTP никто не знает.
3	**Service**	Нормализует данные: имя → `Аня`, email → `anya@mail.ru`. Проверяет: имя не пустое, в email есть @.
4	**Service**	Спрашивает репозиторий: `GetByEmail(ctx, "anya@mail.ru")`.
5	**Repository**	Выполняет SELECT. Строк нет → `sql.ErrNoRows` → возвращает `models.ErrUserNotFound`.
6	**Service**	Видит «не найден» — значит, email свободен. Создаёт `&models.User{Name, Email}` и вызывает `repo.Create`.
7	**Repository**	Выполняет INSERT ... RETURNING, заполняет `ID = 1` и `CreatedAt`.
8	**Service**	Возвращает готового пользователя хендлеру.
9	**Handler**	Ошибки нет → `writeJSON(w, 201, user)`.

Клиент получает:

ответ

```
HTTP/1.1 201 Created
Content-Type: application/json

{"id": 1, "name": "Аня", "email": "anya@mail.ru", "created_at": "2026-10-26T10:00:00Z"}
```

А если отправить тот же email ещё раз?

На шаге 5 репозиторий найдёт пользователя и вернёт его. Сервис увидит, что `existing != nil`, и вернёт `models.ErrEmailTaken`. Хендлер через `handleError` превратит её в `409 Conflict` и `{"error": "email already taken"}`. Каждый слой сделал ровно свою часть.

11. Интерфейсы и тестирование

Вот где слоистая архитектура окупается по-настоящему. Поскольку сервис зависит от интерфейса `UserRepository`, а не от PostgreSQL, мы можем протестировать всю бизнес-логику **без базы данных**. Для этого пишем «фейковый» репозиторий, который хранит пользователей в обычной map:

internal/service/user_test.go

```go
package service

import (
    "context"
    "errors"
    "testing"

    "myapp/internal/models"
)

// fakeRepo — простая реализация UserRepository в памяти.
// Подходит под интерфейс, потому что у неё есть все три метода.
type fakeRepo struct {
    users  map[int64]*models.User
    nextID int64
}

func newFakeRepo() *fakeRepo {
    return &fakeRepo{users: make(map[int64]*models.User), nextID: 1}
}

func (f *fakeRepo) Create(ctx context.Context, u *models.User) error {
    u.ID = f.nextID
    f.nextID++
    f.users[u.ID] = u
    return nil
}

func (f *fakeRepo) GetByID(ctx context.Context, id int64) (*models.User, error) {
    u, ok := f.users[id]
    if !ok {
        return nil, models.ErrUserNotFound
    }
    return u, nil
}

func (f *fakeRepo) GetByEmail(ctx context.Context, email string) (*models.User, error) {
    for _, u := range f.users {
        if u.Email == email {
            return u, nil
        }
    }
    return nil, models.ErrUserNotFound
}

func TestRegister_Success(t *testing.T) {
    svc := NewUserService(newFakeRepo())

    user, err := svc.Register(context.Background(), "  Аня ", "Anya@Mail.ru")
    if err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    if user.Name != "Аня" || user.Email != "anya@mail.ru" {
        t.Errorf("data was not normalized: %+v", user)
    }
}
```

internal/service/user_test.go

```go
func TestRegister_EmailTaken(t *testing.T) {
    svc := NewUserService(newFakeRepo())
    ctx := context.Background()

    if _, err := svc.Register(ctx, "Аня", "anya@mail.ru"); err != nil {
        t.Fatalf("unexpected error: %v", err)
    }
    // Тот же email, но другим регистром — должен считаться занятым
    _, err := svc.Register(ctx, "Аня 2", "ANYA@mail.ru")
    if !errors.Is(err, models.ErrEmailTaken) {
        t.Fatalf("want ErrEmailTaken, got %v", err)
    }
}

func TestRegister_EmptyName(t *testing.T) {
    svc := NewUserService(newFakeRepo())

    _, err := svc.Register(context.Background(), "   ", "a@b.c")
    if !errors.Is(err, models.ErrInvalidInput) {
        t.Fatalf("want ErrInvalidInput, got %v", err)
    }
}
```

Запуск: `go test ./internal/service/...`. Тесты проходят за миллисекунды, не требуют Docker, базы и сети.

Запомни

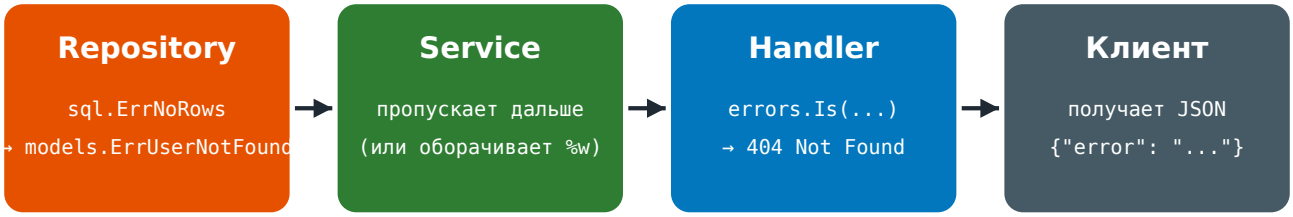
Мы проверили **все бизнес-правила** (нормализацию, уникальность email, пустое имя), не запуская ни базу, ни HTTP-сервер. В «спагетти-коде» из раздела 1 это было бы невозможно. Именно ради этого всё и затевалось.

Совет

Держи интерфейсы маленькими. Если хендлеру нужны только `Register` и `GetUser` — в его интерфейсе только они. Не надо делать один огромный интерфейс на 20 методов «на всякий случай»: его тяжело реализовать в тестах и сложно понять. В Go есть поговорка: *«Чем больше интерфейс, тем слабее абстракция»*.

12. Как ошибки проходят через слои

Ошибки — отличный способ проверить, понимаешь ли ты слои. Каждая ошибка рождается на своём уровне и, поднимаясь вверх, «переводится» на язык следующего слоя.



Ошибка «поднимается» снизу вверх и на каждом этапе говорит на языке своего слоя

Слой	Что делает с ошибкой	Пример
Repository	Переводит ошибки хранилища в доменные. Неожиданные — оборачивает с контекстом.	`sql.ErrNoRows` → `ErrUserNotFound` `fmt.Errorf("select user: %w", err)`
Service	Создаёт ошибки бизнес-правил. Чужие ошибки пропускает или оборачивает, сохраняя исходную через %w.	`return nil, models.ErrEmailTaken` `fmt.Errorf("create user: %w", err)`
Handler	Единственное место, где ошибка становится HTTP-статусом. Внутренние детали логирует, клиенту отдаёт общее сообщение.	`errors.Is(err, ErrUserNotFound)` → 404 неизвестная → 500 + лог

Почему `%w` и `errors.Is`, а не сравнение через ==

Когда сервис пишет `fmt.Errorf("%w: name is required", models.ErrInvalidInput)`, он создаёт **новую** ошибку с подробным текстом, внутри которой «завёрнута» исходная. Простое сравнение `err == models.ErrInvalidInput` вернёт false — это уже другая ошибка. А `errors.Is` разворачивает цепочку и находит `ErrInvalidInput` внутри. Так мы получаем и понятный текст для лога, и возможность определить тип ошибки.

> **Осторожно**
>
> Никогда не отправляй клиенту `err.Error()` для неизвестных ошибок. Там может оказаться имя таблицы, адрес базы или кусок SQL — это подарок для взломщика. Поэтому в handleError для ветки default мы пишем подробности в лог, а клиенту отдаём просто `internal server error`.

13. Частые ошибки новичков

Почти все через это проходят. Хорошая новость: каждую из этих ошибок легко заметить, если знать, что искать.

Ошибка	Почему это плохо	Как правильно
SQL прямо в хендлере	Логика склеена с HTTP и базой. Не протестировать, не переиспользовать.	Хендлер → сервис → репозиторий. SQL живёт только в repository.
`http.ResponseWriter` **или** `*http.Request` **в сервисе**	Сервис привязан к HTTP — для бота или CLI его придётся переписывать.	Сервис принимает обычные значения (строки, числа, структуры) и ctx.
Сервис возвращает статус-код (404, 409)	Это знание из мира HTTP. Сервис не должен знать, как его вызвали.	Сервис возвращает доменную ошибку, handler сам выбирает статус.
Бизнес-правила в репозитории	Правила размазаны по слоям, при смене базы их можно потерять.	Репозиторий только хранит и достаёт. Решения принимает service.
`sql.ErrNoRows` **уходит наверх**	Сервис и хендлер начинают зависеть от database/sql.	Репозиторий переводит её в `models.ErrUserNotFound`.
Глобальная переменная `var db *sql.DB`	Не подменить в тестах, непонятно, кто её использует.	Передавать зависимости через конструкторы New...().
Хендлер вызывает репозиторий напрямую «потому что логики нет»	Завтра логика появится, и её начнут писать в хендлере.	Всегда через сервис, даже если метод сервиса пока в одну строчку.
Один огромный интерфейс на всё	Тяжело реализовать фейк в тестах, непонятно, что реально нужно.	Маленькие интерфейсы в месте использования.
Нет `ctx` **в методах**	Запросы к базе не отменяются, когда клиент ушёл, — сервер тратит ресурсы.	`ctx context.Context` первым аргументом во всех слоях.
Клиенту отдаётся `err.Error()` **от базы**	Утечка внутреннего устройства системы.	Логировать подробно, клиенту — общее сообщение.

А вот это нормально

Ошибка «сервис-прокладка» — когда метод сервиса просто вызывает репозиторий и больше ничего не делает — это **не ошибка**. Да, иногда сервис тонкий. Это нормально: у тебя есть готовое место, куда лягут правила, когда они появятся.

14. Шпаргалка: что куда положить

Когда сомневаешься, в какой слой положить код, — загляни сюда.

Задача	Слой
Прочитать JSON из тела запроса	**Handler**
Достать id из URL и превратить в число	**Handler**
Выбрать HTTP-статус ответа (200, 201, 400, 404, 500)	**Handler**
Отправить JSON-ответ	**Handler**
Проверить, что имя не пустое и email корректный	**Service**
Проверить, что email ещё не занят	**Service**
Захэшировать пароль перед сохранением	**Service**
Посчитать итоговую цену заказа со скидкой	**Service**
Проверить, что пользователь может редактировать этот пост	**Service**
Отправить письмо после регистрации (через интерфейс)	**Service**
Написать SQL-запрос	**Repository**
Превратить `sql.ErrNoRows` в `ErrUserNotFound`	**Repository**
Перевести строки таблицы в структуру Go	**Repository**
Описать структуру `User` или `Order`	**Models**
Объявить `ErrUserNotFound`, `ErrEmailTaken`	**Models**
Подключиться к базе, создать все слои, запустить сервер	**main.go**

15. Задания для практики

Лучший способ закрепить слои — самому пройти через них. Попробуй по порядку:

1. **Список пользователей.** Добавь `GET /users`: метод `List` в репозиторий (подсказка: QueryContext, цикл по rows.Next(), не забудь `defer rows.Close()` и проверку `rows.Err()`), метод в сервис, хендлер и маршрут. Не забудь добавить метод в интерфейсы.
2. **Удаление.** Добавь `DELETE /users/{id}`. Если пользователя нет — должен вернуться 404. Подумай: какой слой узнаёт, что ничего не удалилось? (подсказка: `result.RowsAffected()`).
3. **Новое бизнес-правило.** Добавь поле `Age` и правило «регистрироваться можно с 14 лет». Ответь себе: в какой слой ляжет проверка и почему?
4. **Хранилище в памяти.** Напиши второй репозиторий на map (как fakeRepo в тестах) и подключи его в `main.go` вместо PostgreSQL. Если сервис и хендлер не пришлось менять — поздравляю, ты понял, зачем нужны интерфейсы.
5. **Тест для GetUser.** Проверь два случая: пользователь найден и `ErrUserNotFound`. И третий: id = 0 возвращает `ErrInvalidInput`.
6. **Вторая сущность.** Добавь посты (Post: id, user_id, title, text) со всеми четырьмя слоями. Правило: создать пост можно только от существующего пользователя. Сервису постов понадобятся сразу два репозитория — отличная тренировка.

Итог

Handler переводит с языка HTTP на язык Go и обратно.
Service решает, что должно произойти, по правилам бизнеса.
Repository знает, где и как хранятся данные.
Models — общий словарь, на котором говорят все.

Зависимости идут только сверху вниз, слои общаются через маленькие интерфейсы, а собирает всё main.go. Удачи на курсе!